

LAB Manual for Data Science
LAB-02
Introduction to Basic Python Programming Concepts

Experiments

Loops

Loops are used to execute a block of code repeatedly. Python supports two main loops:

- for loop
- while loop

Step 1: Example (for loop):

```
for i in range(5):  
    print(i)
```

Activity 1:

Write a Python program that uses a for loop to print all numbers from 1 to 10. Additionally, write a program using an if-else statement to check if a given number is positive, negative, or zero.

For loop in a list:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

Looping Through a String:

```
for x in "banana":  
    print(x)
```

The break Statement:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```

The continue Statement:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

While Loop:

```
i = 1
while i < 6:
    print(i)
    i += 1
```

Activity 2:

Write a Python program that:

1. Has a list of fruits: ["apple", "banana", "cherry", "mango", "orange"].
 - Use a **for loop** to print each fruit in the list.
 - If the fruit is "cherry", use a **break statement** to stop the loop.
2. Then, take a string: "programming".
 - Use a **for loop** to print each character in the string.
 - Use a **continue statement** to skip printing the letter 'm'.

Note: Attempt with while loop and for loop.

Functions:

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.

```
def my_function():
    print("Hello from a function")
```

Calling a Function

```
def my_function():
    print("Hello from a function")

my_function()
```

Arguments

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

```
def my_function(fname):
    print(fname + " World!")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

```
def my_function(*kids):
    print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

Return Values

```
def my_function(x):
    return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

Activity 3:

Write a Python program using **functions** to perform the following tasks:

1. Create a function named `student_details(name, age)` that prints the student's name and age.
2. Create another function named `calculate_average(*marks)` that accepts any number of marks using *args and returns the average marks.
3. Create a third function named `get_grade(average)` that returns the grade based on the average:
 - 90 and above → A+
 - 80–89 → A
 - 70–79 → B
 - Below 70 → C
4. In the main program:
 - Call all the above functions.
 - Display the student's name, age, average marks, and grade.

Arrays:

- An array is a special variable, which can hold more than one value at a time.
- An array can hold many values under a single name, and you can access the values by referring to an index number.

```
cars = ["Ford", "Volvo", "BMW"]

x = cars[0]      #Accessing array element

x = len(cars)    #Length of an array

for x in cars:   #Looping array elements
    print(x)

cars.append("Honda")  #Adding array elements
cars.pop(1)           #Removing array elements
cars.remove("Volvo")
```

Activity 4:

Write a Python program that performs the following operations using **arrays (lists)**:

1. Create a list of integers entered by the user.
2. Write a function `find_second_largest(numbers)` that finds and returns the **second largest** number in the list.
3. Write another function `reverse_array(numbers)` that returns the **reversed list** without using the built-in `reverse()` or slicing (`[::-1]`).
4. In the main program:
 - Accept at least 5 integers from the user.
 - Call both functions.
 - Display the original list, the second largest number, and the reversed list.

Python Classes/Objects

- Python is an object oriented programming language.
- Almost everything in Python is an object, with its properties and methods.
- A Class is like an object constructor, or a "blueprint" for creating objects.

```
#Create class
class MyClass:
    x = 5

#Create object
p1 = MyClass()
print(p1.x)
```

The `__init__()` Method

- All classes have a method called `__init__()`, which is always executed when the class is being initiated.
- Use the `__init__()` method to assign values to object properties, or other operations that are necessary to do when the object is being created.

```
class Person:
    def __init__(self, name, age): #self → refers to the current object.
        self.name = name
        self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

Activity 5:

Create a Python program using **classes** that performs the following tasks:

1. Define a class named Student with attributes: name, age, and marks.

2. Include a **constructor** (`__init__`) to initialize these attributes.
3. Create a method `display_info()` to print the student's details.
4. Create another method `calculate_grade()` that returns a grade based on marks:
 - 90 and above → A+
 - 80–89 → A
 - 70–79 → B
 - Below 70 → C
5. In the main program, create **two student objects**, call both methods, and display their information with grades.

Lab Assignment

Task 1:

1. Write a program that takes a list of student IDs (integers) entered by the user.
2. Use a **for loop** to count how many IDs were scanned.
3. If any ID appears **more than once**, display a warning message for that student.
4. Stop scanning if the ID entered is 0 (use `break`).
5. Finally, print total students present.

Hint: Use loops and if conditions to detect duplicates.

Task 2:

1. Take a password input from the user.
2. Use a **while loop** to re-ask the user until the password is at least **8 characters long**.
3. Then use a **for loop** to check:
 - Must contain at least one uppercase letter
 - Must contain at least one number
 - Must contain one special character (`!@#$%&*`)
4. If any condition fails, show a hint to the user to improve the password.

Hint: Use `any()` and string methods like `.isupper()` and `.isdigit()`.

Task 3:

1. Create a function `add_student(name, *marks)` that stores student data (name and marks) in a list.
2. Create another function `calculate_average(*marks)` that returns the average marks.
3. Create a third function `assign_grade(average)` that returns grades based on the scale.
4. Allow the teacher to enter data for **5 students** using loops.
5. Display all students' names with their averages and grades in a formatted way.

Hint: Combine loops, functions, and *args for flexibility.

Task 4:

1. Take 7 temperature readings (integers) for a week and store them in a list.
2. Write a function `get_max_min(temps)` that returns both **maximum and minimum** temperatures.
3. Write another function `above_average(temps)` that prints all days with above-average temperature.
4. Do not use built-in functions like `max()` or `min()` — calculate manually using loops.

Hint: Calculate total, divide by length, and compare each element.

Task 5:

1. Create a class `Book` with attributes: `title`, `author`, and `is_borrowed`.
2. Use a constructor to initialize these.
3. Create methods:
 - `borrow_book()` → marks the book as borrowed
 - `return_book()` → marks the book as available
 - `display_info()` → shows book details and status
4. In main, create a **list of 5 books**, let user borrow or return books using input and loops.

Hint: Manage book status using Boolean values.

Task 6:

1. Create a class `Employee` with attributes: `name`, `department`, and `monthly_scores` (list).
2. Create methods:
 - `add_score(score)` → adds monthly score to the list
 - `average_score()` → calculates and returns average
 - `performance_level()` → returns “Excellent”, “Good”, “Average”, or “Poor” based on average
3. In main:
 - Create 3 employee objects
 - Add random monthly scores using a loop
 - Display each employee’s average and performance level

Hint: Use list operations and class methods together.